

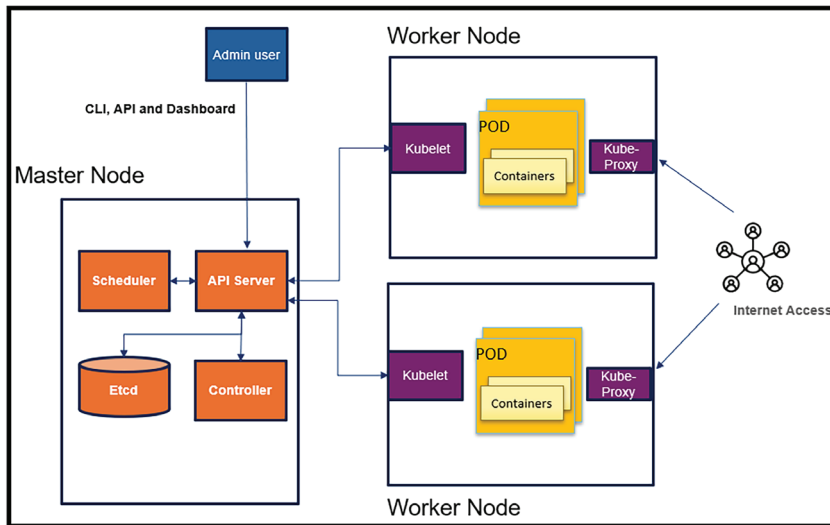


Figure 2: Kubernetes architecture

- rescheduling of failed containers.
- Controlled exposure of network ports to systems outside the cluster.
- Limits hardware usage to required resources only.
- Self-healing, with auto-placement, auto-restart, auto-replication and auto-scaling.

Here's a quick overview of Kubernetes architecture (Figure 2):

- Kubernetes users communicate with the API server and apply the desired state.
- The master node is responsible for managing the Kubernetes cluster. Master nodes actively enforce the desired state on worker nodes.
- Worker nodes support communication between containers.
- Pod is the scheduling unit.
- Kubelet is the agent that receives definitions from the master and implements them.
- Kube-proxy is the network proxy in each node and listens to the API server.
- Etcd is the distributed key value store for cluster state and configuration. It can be run in the master or outside.
- Master, worker node and etcd can be one or more.

End-to-end flow of microservices deployment

In microservices architecture, each service is independently developed, packaged, and deployed. Containerisation (using tools like Docker) ensures that each microservice runs in an identical environment, while Kubernetes takes care of orchestration—handling deployment, scaling, and management of these containerised workloads.

Here's how microservices deployment flows from development through containerisation to deployment in a Kubernetes cluster (Figure 3):

Develop microservices: Developers write code for discrete business functions (each microservice).

Container image build: Using tools like Docker, source code is packaged with all dependencies.

Container registry: Image artifacts are saved in registries (e.g., Docker Hub) for storage and distribution.

CI/CD pipeline: Automated processes build, test, and trigger deployments.

Kubernetes cluster: The CI/CD pipeline deploys containerised applications into a Kubernetes cluster.

Control plane and worker nodes: The cluster's control plane (with API servers, schedulers, etc) handles

orchestration while worker nodes host the actual application pods.

API gateway/Ingress controller:

Incoming client requests are managed through an API gateway or ingress controller that looks up the appropriate service endpoints.

Microservices (Service A/B/C):

Independent microservices—each aligned with a business unit—are deployed in pods across the nodes.

Service discovery: Kubernetes provides internal DNS and load balancing so that service endpoints are dynamically discovered and routed.

Best practices for implementing microservices

Business capability-based design:

Microservices architecture typically needs teams that specialise in particular business capabilities with different requirements, such as order approval, shipping, delivery and returns management, which are all part of order fulfillment.

Product mindset: Enterprises should aim to focus not on more projects, but on products. Services should be developed as independent products that are well documented, with each responsible for a single business capability.

Data store for each microservice:

The same data store should not be used for all microservices. Each microservices team should select the database that is best suited to its service.

Code maturity: All code in a microservice should have a similar degree of maturity and stability. To add or rewrite code that works well in a microservice, the best way is usually to create a new microservice for the new or altered code and leave the existing microservice where it is. This allows teams to iteratively release and test the new code until it is error-free and maximally efficient.

Containerisation: There should be only one microservice per container. Minimal base images should be generated to protect from unintentional packages/libraries. The container image should be small (removing redundant